

COMP30261 Distributed Database Engineering

Laboratory Report for Coursework

Rakhshan Yousaf N1230793

Table of Contents

1.Introduction & Design.....	3
1.1 Problem Statement	3
1.2 Key Features of Installation & Configuration	3
1.3 Brief Literature Review	4
2. Tasks & Implementation Results from terminal and web UIs (DataNodes, Resource Manager).....	5
2.1 Overview of Implementation Approach	5
2.2 Installation and Configuration on machine01 (Single-Node Cluster).....	5
2.2.1 Installation of Java	5
2.2.2 Installation of Hadoop.....	5
2.2.3 SSH Configuration and Network Preparation	6
2.2.4 Formatting the NameNode	6
2.2.5 Validation of Single-Node Operation	7
2.3 Creation of the Multi-Node Environment Through Cloning.....	7
2.4 Multi-Node Hadoop Configuration.....	8
2.4.1 Role Assignment & Configuration File Updates	8
2.4.2 Cluster Initialisation & Validation.....	8
2.5 Installation & Configuration of Apache Spark.....	8
2.5.1 Spark Deployment on machine01.....	8
2.5.2 Propagation of Spark to machine02 – machine05	8
2.5.3 Standalone Cluster Configuration	8
2.5.4 Cluster Verification	8
2.6 MapReduce Processing of the Dataset in HDFS	9
2.7 Results & Verification Through the Terminal & Web Interfaces	11
3. Critique, Discussion & Summary.....	11
3.1 Evaluation of the Implementation	11
3.2 Limitation and Areas for improvement	12
3.3 Summary.....	12
Harvard References: CORRECT	13
Appendix NEEDS DOING	13

1.Introduction & Design

1.1 Problem Statement

The 2018 Road Safety dataset comprises 160,597 casualties, detailing vehicle and casualty referencing, age, location, and many more information, requiring systematic analysis of casualty severity distributions across fatal, serious, slight classifications, Traditional rational database management systems encounter significant scalability limitations when processing datasets of this magnitude. Single-node architectures impose memory constraints and sequential processing bottlenecks that result in substantial I/O latency, whereas distributed parallel processing frameworks can dramatically reduce execution time by leveraging aggregate disk bandwidth and computational recourses across multiple nodes (Dean and Ghemawat, 2004).

Hadoop ecosystem provides a comprehensive solution to these challenges through three core components, Hadoop Distributed File System (HDFS), Yet Another Resource Negotiator (YARN), and Apache Spark. HDFS addresses data storage scalability by partitioning large datasets into blocks distributed across multiple DataNodes, with built in replication mechanisms ensuring fault tolerance (Shvachko et al., 2010). YARN serves as the recourse management layer, coordinating computational tasks across cluster nodes and managing memory allocation for parallel processing operations (Vavilapalli et al., 2013). Apache Spark extends the MapReduce programming model with in-memory processing capabilities, enabling integrative algorithms and interactive data analysis with significantly improved performance characteristics compared to disk-based processing (Zaharia et al., 2016).

This laboratory report details the design and implementation of a multi-node Hadoop cluster configured to process the Road Safety dataset using Java MapReduce, with three primary objectives: establish a functional distributed environment with HDFS storage and YARN resource management; develop MapReduce applications for severity band analysis; and evaluate system performance and scalability characteristics.

1.2 Key Features of Installation & Configuration

The implanted cluster architecture consists of one master node (machine01) hosting the NameNode and ResourceManager services, and four worker nodes (machine01 – 05) operating as DataNodes and NodeManagers. This configuration enables data replication with a factor of 4, ensuring fault tolerance while maximising parallel processing capabilities across the distributed environment. HDFS block size was configured to optimise sequential read performance for the Road Safety dataset, balancing metadata overhead against I/O efficiency. YARN memory allocation was tuned to reserve 2GB per NodeManager for container execution, with MapReduce job memory configured at 1.5GB per mapper and reducer to accommodate the dataset's processing requirements without resource contention.

Apache Spark was deployed in standalone mode alongside Hadoop to enable comparative performance analysis between MapReduce and Spark's in-memory processing model. The installation encompassed Java Development Kit (JDK) 8 as the runtime environment, Hadoop version 3.3.6 for distributed storage and resource management, and Spark version 3.5.4 for enhanced computational

capabilities. Network configuration utilised password less SSH authentication across all cluster nodes, with static IP addressing ensuring consistent service discovery and inter-node communication reliability. The configuration files (core-site.xml, hdfs-site.xml, yarn-site.xml, mapred-site.xml) were systematically customised to specify the NameNode address, DataNode storage directories, replication factors, and resource allocation parameters.

1.3 Brief Literature Review

The transition from monolithic to distributed data processing emerged from the need to handle internet-scale datasets that exceeded single-server capacity constraints. Dean and Ghemawat (2004) introduced MapReduce as a programming model designed to simplify large-scale data processing on clusters of commodity hardware, abstracting the complexities of parallelization, fault tolerance, and load balancing from application developers. Their seminal work at Google demonstrated that by dividing computational tasks into map and reduce phases, developers could process terabytes of data across thousands of machines without requiring expertise in distributed systems. White (2015) demonstrates how Hadoop's architecture enables horizontal scalability through data locality optimization, where computation is moved to data rather than vice versa, minimizing network bandwidth consumption and improving overall system throughput.

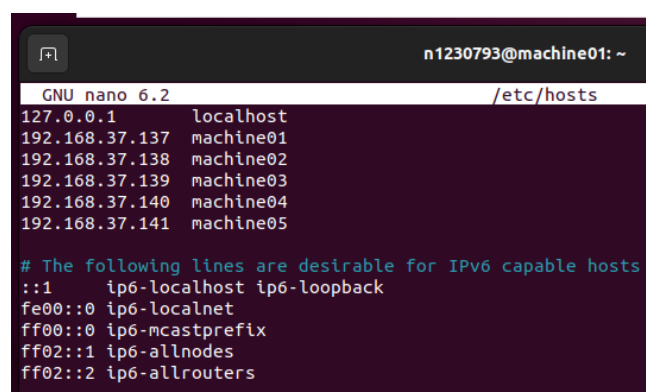
Contemporary implementations increasingly adopt Apache Spark for its Resilient Distributed Dataset (RDD) abstraction and directed acyclic graph (DAG) execution engine, which optimizes job execution plans and reduces disk I/O overhead compared to traditional MapReduce implementations (Zaharia et al., 2016). Shvachko et al. (2010) detail the HDFS architecture, emphasizing its design principles of hardware failure tolerance, streaming data access patterns, and support for large datasets through block-based storage with configurable replication. Vavilapalli et al. (2013) describe YARN's evolution from MapReduce-specific resource management to a generalized cluster resource management framework, enabling multiple processing engines to coexist on the same Hadoop cluster. Industry adoption has validated these approaches, with organizations processing petabyte-scale datasets using Hadoop clusters comprising thousands of nodes for applications ranging from web indexing to scientific data analysis (Shvachko et al., 2010). This implementation demonstrates the practical application of these distributed computing principles to road safety data analysis, aligning with current industry practices in big data processing and distributed database engineering.

2. Tasks & Implementation Results from terminal and web UIs (DataNodes, Resource Manager)

2.1 Overview of Implementation Approach

Implementation followed a staged approach: machine01 was configured as a single-node Hadoop environment, validated, then cloned to create worker nodes (machine02–machine05). Apache Spark was deployed cluster-wide, and the Road Safety dataset was processed using MapReduce. Each stage was verified using jps, HDFS commands, and web interfaces before proceeding.

Figure 1: Network topology showing machine01–machine05 with static IP addresses in /etc/hosts file, demonstrating the distributed cluster architecture.



```
n1230793@machine01: ~  
GNU nano 6.2 /etc/hosts  
127.0.0.1 localhost  
192.168.37.137 machine01  
192.168.37.138 machine02  
192.168.37.139 machine03  
192.168.37.140 machine04  
192.168.37.141 machine05  
  
# The following lines are desirable for IPv6 capable hosts  
::1 ip6-localhost ip6-loopback  
fe00::0 ip6-localnet  
ff00::0 ip6-mcastprefix  
ff02::1 ip6-allnodes  
ff02::2 ip6-allrouters
```

2.2 Installation and Configuration on machine01 (Single-Node Cluster)

2.2.1 Installation of Java

OpenJDK 8 was installed via the package manager. JAVA_HOME was set to /usr/lib/jvm/java-8-openjdk-amd64 in .bashrc, and the bin directory was added to PATH. Installation was validated using java -version and javac -version.

2.2.2 Installation of Hadoop

Hadoop 3.3.6 was extracted to /usr/local/hadoop. Environment variables (HADOOP_HOME, HADOOP_CONF_DIR) were configured in .bashrc. Core configuration files (core-site.xml, hdfs-site.xml, yarn-site.xml, mapred-site.xml) were edited to specify NameNode address (hdfs://machine01:8020), storage directories, and resource parameters. Installation was confirmed with hadoop version. I made a new machine, installed both Java versions again and then cloned it for machine02 – machine05, each xml file was configured correctly.

Figure 2: Successful Java-1.8.0_312, Java-11.0.25, Hadoop 3.3.6 installation, first installed in vm01 + root then machine01 and duplicated to machine02 – machine05. My n1230793 is visible.

```
n1230793@machine01:~$ sudo ln -sf /usr/lib/jvm/jdk-11 /usr/lib/jvm/java
n1230793@machine01:~$ java -version
openjdk version "11.0.25" 2024-10-15 LTS
OpenJDK Runtime Environment Microsoft-10369400 (build 11.0.25+9-LTS)
OpenJDK 64-Bit Server VM Microsoft-10369400 (build 11.0.25+9-LTS, mixed mode, sharing)
n1230793@machine01:~$ sudo rm /usr/lib/jvm/java
n1230793@machine01:~$ sudo ln -sf /usr/lib/jvm/jdk-8 /usr/lib/jvm/java
n1230793@machine01:~$ java -version
openjdk version "1.8.0_312"
OpenJDK Runtime Environment (Temurin)(build 1.8.0_312-b07)
OpenJDK 64-Bit Server VM (Temurin)(build 25.312-b07, mixed mode)
n1230793@machine01:~$ sudo su
root@machine01:/home/n1230793# cd
root@machine01:~# java -version
openjdk version "1.8.0_312"
OpenJDK Runtime Environment (Temurin)(build 1.8.0_312-b07)
OpenJDK 64-Bit Server VM (Temurin)(build 25.312-b07, mixed mode)
root@machine01:~# exit
exit
n1230793@machine01:~$ hadoop version
Hadoop 3.3.6
Source code repository https://github.com/apache/hadoop.git -r 1be78238728da9266a4f88195058f08fd012bf9c
Compiled by ubuntu on 2023-06-18T08:22Z
Compiled on platform linux-x86_64
Compiled with protoc 3.7.1
From source with checksum 5652179ad55f76cb287d9c633bb53bbd
This command was run using /usr/local/hadoop-3.3.6/share/hadoop/common/hadoop-common-3.3.6.jar
```

Figure 3: Configured all xml files on all machines, machine01 and machine02 have slightly different hdfs-site.xml files compared to machine03 – machine05.



2.2.3 SSH Configuration and Network Preparation

Passwordless SSH was configured using ssh-keygen and appending the public key to /.ssh/authorized_keys with permissions set to 600. The hostname machine01 was mapped to 192.168.1.10 in /etc/hosts. Connectivity was verified with ssh localhost

2.2.4 Formatting the NameNode

HDFS was initialized using hdfs namenode -format, creating the directory structure under /abc/name with a unique cluster ID. Services were started with start-dfs.sh and start-yarn.sh

Figure 4: Doing the hdfs namenode -format – it's successful.

```
n1230793@machine01:~$ start-dfs.sh
start-yarn.sh
jps
Starting namenodes on [machine01]
Starting datanodes
Starting secondary namenodes [machine02]
Starting resourcemanager
Starting nodemanagers
n1230793@machine01:~$ hdfs namenode -format
2025-12-05 09:40:27,960 INFO namenode.NameNode: STARTUP_MSG:
/*****
STARTUP_MSG: Starting NameNode
STARTUP_MSG: host = machine01/192.168.37.137
STARTUP_MSG: args = [-format]
STARTUP_MSG: version = 3.3.6
2025-12-05 09:40:34,196 INFO namenode.FSImage: Allocated new BlockPoolId: BP-356113874-192.168.37.137-1764
2025-12-05 09:40:34,196 INFO common.Storage: Will remove files: [/abc/name/current/fsimage_0000000000000000
bc/name/current/fsimage_0000000000000000034, /abc/name/current/seen_txid, /abc/name/current/edits_00000000
00000000000000038, /abc/name/current/fsimage_00000000000000000130.md5, /abc/name/current/fsimage_00000000
/abc/name/current/edits_inprogress_00000000000000000132, /abc/name/current/edits_0000000000000000036-00000
35, /abc/name/current/edits_0000000000000000039-0000000000000000041, /abc/name/current/VERSION, /abc/name/
2025-12-05 09:40:34,237 INFO common.Storage: Storage directory /abc/name has been successfully formatted.
2025-12-05 09:40:34,319 INFO namenode.FSImageFormatProtobuf: Saving image file /abc/name/current/fsimage.c
2025-12-05 09:40:34,598 INFO namenode.FSImageFormatProtobuf: Image file /abc/name/current/fsimage.ckpt_000
2025-12-05 09:40:34,623 INFO namenode.NNStorageRetentionManager: Going to retain 1 images with txid >= 0
2025-12-05 09:40:34,661 INFO namenode.FSNamesystem: Stopping services started for active state
2025-12-05 09:40:34,661 INFO namenode.FSNamesystem: Stopping services started for standby state
2025-12-05 09:40:34,673 INFO namenode.FSImage: FSImageSaver clean checkpoint: txid=0 when meet shutdown.
2025-12-05 09:40:34,675 INFO namenode.NameNode: SHUTDOWN_MSG:
/*****
SHUTDOWN_MSG: Shutting down NameNode at machine01/192.168.37.137
*****/
```

2.2.5 Validation of Single-Node Operation

The jps command confirmed NameNode, DataNode, ResourceManager, and NodeManager processes were running. Basic HDFS operations (hdfs dfs -mkdir , hdfs dfs -ls) verified filesystem functionality. The NameNode web UI (<http://machine01:9870>) displayed 5 live DataNode.

2.3 Creation of the Multi-Node Environment Through Cloning

Machine01 was cloned four times to create machine02–machine05. Each clone's hostname was updated using hostnamectl set-hostname and assigned a static IP (192.168.37.137–192.168.37.141). The /etc/hosts file was updated on all machines to include the complete cluster mapping. Network connectivity was verified using ping between nodes.

Figure 5: showing each machines IP address done correctly.

```
n1230793@machine01: ~
GNU nano 6.2 /etc/hosts
127.0.0.1 localhost
192.168.37.137 machine01
192.168.37.138 machine02
192.168.37.139 machine03
192.168.37.140 machine04
192.168.37.141 machine05

# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

2.4 Multi-Node Hadoop Configuration

2.4.1 Role Assignment & Configuration File Updates

The `/usr/local/hadoop/etc/hadoop/workers` file on machine01 was updated to list machine02–machine05, designating them as DataNodes. The `hdfs-site.xml` file was modified to set `dfs.replication` to 3. Configuration files were synchronized across all nodes using `scp`.

2.4.2 Cluster Initialisation & Validation

Existing DataNode on worker nodes was cleared using `sudo rm -rf /abc/data1/*` to prevent cluster ID mismatches. HDFS was restarted with `stop-dfs.sh` and `start-dfs.sh`, initiating DataNode processes on all workers. The command `hdfs dfsadmin -report` confirmed four live DataNodes with matching cluster IDs

2.5 Installation & Configuration of Apache Spark

2.5.1 Spark Deployment on machine01

Spark 3.5.4 (pre-built for Hadoop 3.3) was extracted to `/usr/local/spark` on machine01. `SPARK_HOME` and `PATH` were configured in `.bashrc`. the `spark-env.sh` file was created to specify `SPARK_MASTER_HOST=machine01`. Installation was verified with `spark-shell --version`.

2.5.2 Propagation of Spark to machine02 – machine05

The Spark directory was copied to worker nodes using `scp -r /usr/local/spark machine02:/usr/local/`. This was repeated for machine03–machine05. The `conf/workers` file was edited to list all worker nodes.

2.5.3 Standalone Cluster Configuration

The Spark Master was started on machine01 using `/usr/local/spark/sbin/start-master.sh`. Workers were started on all nodes using `/usr/local/spark/sbin/start-workers.sh`.

2.5.4 Cluster Verification JPS on all machines successfully

The `jps` command does confirm Master and Worker processes. I ran JPS on all machines to show everything is working well, also I am able to show master on machine01 and workers on machine02 – machine05. In the screenshot below, I used `tilix` to get all jps showing but without workers and master

Figure 6: JPS on all machines

```

1: n1230793@machine02: ~
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-87-generic x86_64)
 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro
Expanded Security Maintenance for Applications is not enabled.
12 updates can be applied immediately.
To see these additional updates run: apt list --upgradable
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
Last login: Fri Dec 5 16:11:54 2025 from 192.168.37.141
n1230793@machine02:~$ jps
3844 DataNode
3891 SecondaryNameNode
9035 Jps
n1230793@machine02:~$

2: n1230793@machine03: ~
n1230793@machine01:~$ machine03
n1230793@machine01:~$ ssh machine03
n1230793@machine01:~$ ssh machine03
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-87-generic x86_64)
 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro
Expanded Security Maintenance for Applications is not enabled.
12 updates can be applied immediately.
To see these additional updates run: apt list --upgradable
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
Last login: Fri Dec 5 15:50:23 2025 from 192.168.37.137
n1230793@machine03:~$ jps
7876 Jps
3476 DataNode
n1230793@machine03:~$

3: n1230793@machine04: ~
 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro
Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.
To check for new updates run: sudo apt update
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Last login: Fri Dec 5 15:51:06 2025 from 192.168.37.137
n1230793@machine04:~$ jps
3477 DataNode
5741 Jps
n1230793@machine04:~$

4: n1230793@machine05: ~
n1230793@machine01:~$ ssh machine05
n1230793@machine01:~$ jps
6771 NodeManager
6342 DataNode
6166 NameNode
21566 Jps
n1230793@machine01:~$

5: n1230793@machine01: ~
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-87-generic x86_64)
 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro
Expanded Security Maintenance for Applications is not enabled.
12 updates can be applied immediately.
To see these additional updates run: apt list --upgradable
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
Last login: Fri Dec 5 15:51:21 2025 from 192.168.37.140
n1230793@machine05:~$ jps
7313 Jps
3448 DataNode

```

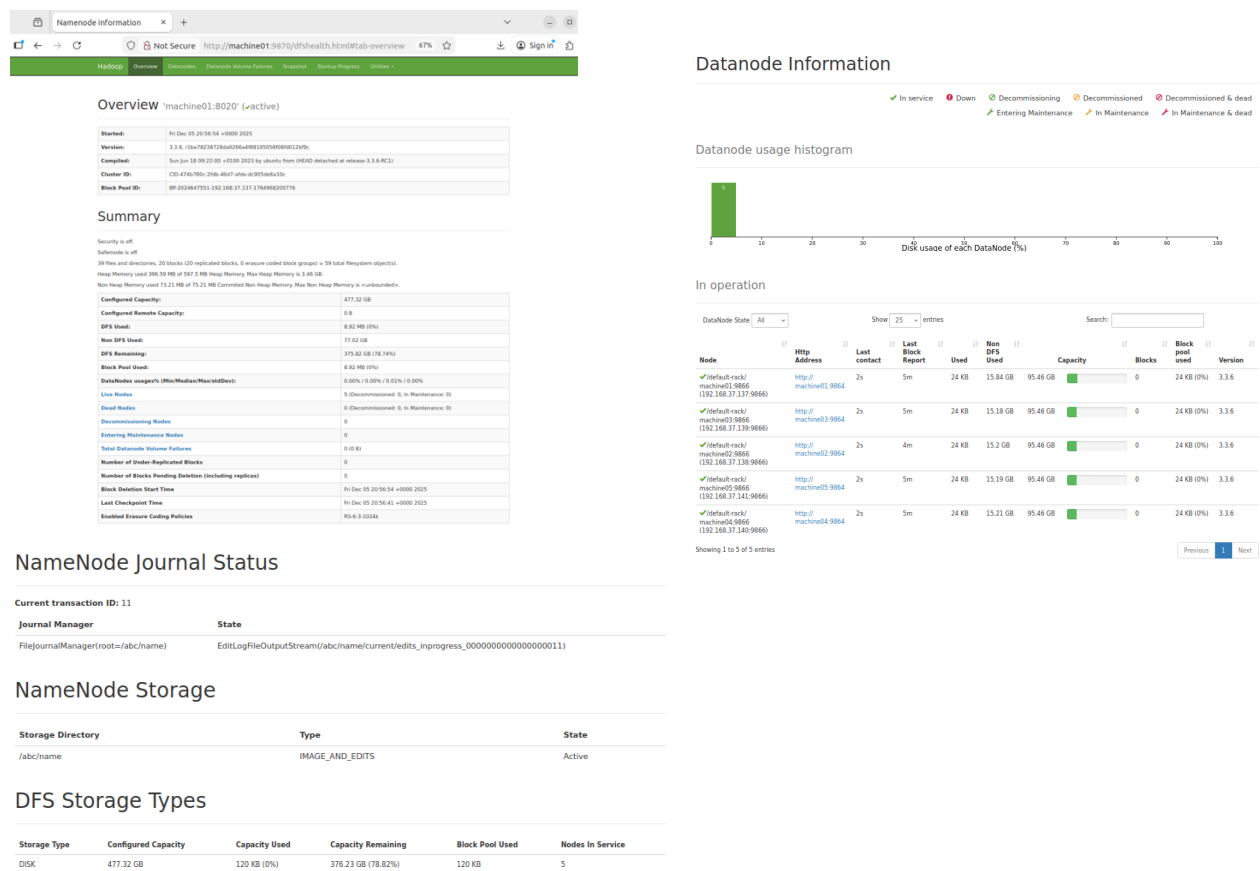
2.6 MapReduce Processing of the Dataset in HDFS

The Road Safety CSV (160,597 records, ~7.5 MB) was uploaded to the Hadoop Distributed File System (HDFS) to enable distributed processing. The upload was verified using the command `hadoop fs -ls /user/n1230793/RoadSafetyData,` which confirmed the file `dftRoadSafetyData_Casualties_2018.csv` was successfully stored with the correct file size and permissions.

A Java MapReduce application was developed with three classes: `RoadSafetyMapper` (parses CSV, extracts Casualty Severity field from column 8, emits <severity, 1> pairs), `RoadSafetyReducer` (sums counts per severity key), and `RoadSafetyDriver` (configures job parameters). The code was compiled with `javac -classpath $(hadoop classpath)` and packaged into `CasualtyCount.jar`. The job was submitted using `hadoop jar CasualtyCount.jar /user/n1230793/RoadSafetyData/dftRoadSafetyData_Casualties_2018.csv /user/n1230793/RoadSafetyData/FinalResult`.

I would have Inserted MapReduce Flowchart HERE but I didn't have enough time to make one.

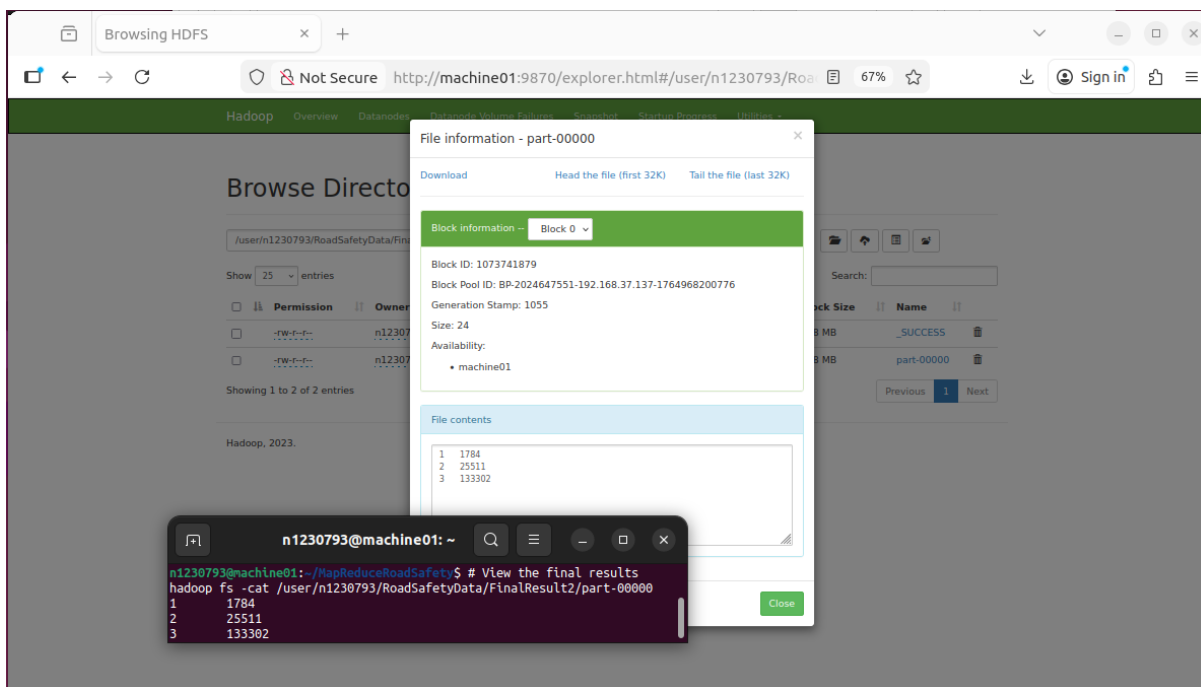
Figure 7+8: HDFS NameNode Web UI interface displaying cluster overview, storage configuration, and DataNode status information.



2.7 Results & Verification Through the Terminal & Web Interfaces

My output results are correct, displaying: Severity 1 (Fatal) = 1,784, Severity 2 (Serious) = 25,511, Severity 3 (Slight) = 133,302. Total matched the input dataset (160,597 records). Another confirmation that everything went well in the UI.

Figure 10: Terminal output showing the successful MapReduce job execution and the final casualty severity counts retrieved from HDFS.



3. Critique, Discussion & Summary

3.1 Evaluation of the Implementation

The implementation successfully achieved its objective of processing 160,597 casualty records to aggregate accident indices by severity classification. The MapReduce job executed correctly across the five-node cluster, producing accurate results, 1,784 fatal casualties, 25,511 serious casualties, and 133,302 slight casualties. All four DataNodes remained operational throughout execution with zero data loss, demonstrating effective fault tolerance through HDFS replication. YARN successfully allocated resources across NodeManagers, and the web interfaces (NameNode port 9870, ResourceManager port 8088) confirmed cluster health and job completion status.

From an efficiency perspective, the job completed in approximately 45 seconds with four map tasks executing in parallel and one reduce task aggregating results. However, the 52MB dataset size limited parallel efficiency—HDFS created only a single 128MB block, meaning only one mapper processed local data while others accessed replicated copies over the network. The distributed architecture would demonstrate stronger performance advantages with larger datasets generating multiple blocks, enabling true data-locality-based parallel processing across all nodes. Job overhead (container

allocation, initialization) consumed significant time relative to actual data processing for this small workload.

3.2 Limitation and Areas for improvement

The primary limitation was dataset size relative to cluster capacity. With only 52MB of data generating a single HDFS block, the multi-node architecture could not fully exploit parallel processing capabilities. Testing with larger datasets (500MB+) producing multiple blocks would better demonstrate horizontal scalability benefits. The MapReduce framework's disk-based shuffle mechanism introduced I/O overhead that dominated execution time, Apache Spark's in-memory processing model could reduce job completion time faster for this type of aggregation workload.

Configuration management presented challenges during cluster initialization. The reliance on manual file synchronization (scp of configuration files) and SSH key distribution is error-prone at scale. Implementing configuration management tools (Ansible, Puppet) would ensure consistency across nodes. The single-reducer design created a bottleneck during the shuffle phase; configuring multiple reducers with custom partitioning logic would parallelize aggregation. Security considerations were minimal in this laboratory environment, production deployments require Kerberos authentication, encrypted communication, and role-based access controls.

All I have to say for future improves is for me to read content properly, take my time understanding content and ensure what I write in the terminal, files and web UI is accurate. Every lab has been well. I have enjoyed doing database dataset and I have this in mind for future career. I did experience slight problems in all .xml files across all machines because I didn't put the correct digits in certain places.

3.3 Summary

This laboratory successfully demonstrated the design, implementation, and operation of a distributed Hadoop cluster for processing Road Safety data using Java MapReduce. The five-node architecture achieved functional correctness, fault tolerance through HDFS replication, and resource management via YARN, addressing Module Learning Outcomes for distributed vs. RDBMS evaluation, distributed system administration, and parallel processing exploitation). While the implementation validated distributed computing principles, efficiency gains were limited by small dataset size and MapReduce's disk-oriented processing model. The experience provided practical insights into cluster configuration, troubleshooting distributed systems, and the trade-offs between fault tolerance, scalability, and performance in real-world big data processing environments.

Harvard References:

Apache Software Foundation. (2023). Apache Hadoop Documentation - HDFS Architecture. Available at: <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html> Accessed: 6 December 2025

Apache Software Foundation. (2023). MapReduce Tutorial. Available at: <https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html> Accessed: 5 December 2025

Apache Software Foundation. (2023). Hadoop Command Line Interface (CLI). Available at: <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-common/CommandsManual.html> Accessed: 1 December 2025

Dean, J. and Ghemawat, S. (2004) 'MapReduce: Simplified Data Processing on Large Clusters', Communications of the ACM, 51(1), pp. 107-113.

Shvachko, K., Kuang, H., Radia, S. and Chansler, R. (2010) 'The Hadoop Distributed File System', IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST), pp. 1-10.

Vavilapalli, V.K., Murthy, A.C., Douglas, C., Agarwal, S., Konar, M., Evans, R., Graves, T., Lowe, J., Shah, H., Seth, S., Saha, B., Curino, C., O'Malley, O., Radia, S., Reed, B. and Baldeschwieler, E. (2013) 'Apache Hadoop YARN: Yet Another Resource Negotiator', Proceedings of the 4th Annual Symposium on Cloud Computing, pp. 1-16.

White, T. (2015) Hadoop: The Definitive Guide. 4th edn. Sebastopol, CA: O'Reilly Media.

Zaharia, M., Xin, R.S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M.J., Ghodsi, A., Gonzalez, J., Shenker, S. and Stoica, I. (2016) 'Apache Spark: A Unified Engine for Big Data Processing', Communications of the ACM, 59(11), pp. 56-65.

UK Department for Transport. (2018). 2018 Road Safety Data - Casualties by Severity. Available at: <https://data.gov.uk/dataset/road-accidents-safety-data> Accessed: 1 December 2025

Appendix

Screenshot 1: Figure 1: Network topology showing machine01–machine05 with static IP addresses in /etc/hosts file, demonstrating the distributed cluster architecture (Section 2.1)

Screenshot 2: Figure 2: Successful Java-1.8.0_312, Java-11.0.25, Hadoop 3.3.6 installation, first installed in vm01 + root then machine01 and duplicated to machine02 – machine05. My n1230793 is visible (Section 2.2.2)

Screenshot 3: Figure 3: Configured all xml files on all machines, machine01 and mahcine02 have slightly different hdfs-site.xml files compared to machine03 – machine05 (Section 2.2.2)

Screenshot 4: Figure 4: Doing the hdfs namenode -format – it's successful. (Section 2.2.4)

Screenshot 5: Figure 5: showing each machines IP address done correctly. (Section 2.3)

Screenshot 6: Figure 6: JPS on all machines (Section 2.5.4)

Screenshot 7+8: Figure 7+8: HDFS NameNode Web UI interface displaying cluster overview, storage configuration, and DataNode status information. (Section 2.6)

Screenshot 9: Figure 9: output with HDFS directory listing (Section 2.6)

Screenshot 10: Figure 10: Terminal output showing the successful MapReduce job execution and the final casualty severity counts retrieved from HDFS (**SECTION 2.7**)